📚 PROJECTE: Sistema de Gestió de Biblioteca Digital

Mòdul: M0613-B2 - PHP orientat a objectes

Lliurament: link al repositori Git + vídeo de demostració (2')

🎯 OBJECTIUS D'APRENENTATGE

Desenvolupar un sistema complet de gestió de biblioteca, amb interfície web, que posi en pràctica tots els conceptes de Programació Orientada a Objectes (POO) vistos a classe:

- ✅ Classes abstractes i herència
- ✅ Interfícies i implementació
- ✅ Traits i reutilització de codi
- ✅ Visibilitat de propietats i mètodes
- ✅ Mètodes màgics (`__toString`, `__call`, `__get`, `__set`, `__sleep`, `__wakeup`)
- ✅ Gestió d'errors amb excepcions personalitzades
- ✅ Namespaces i autoloading PSR-4 amb Composer
- ✅ Serialització i persistència d'objectes a la sessió
- ✅ Característiques de PHP 8: typed properties, promoted properties, union i nullable types, `match`

0️⃣ ESTRUCTURA DEL PROJECTE I AUTOLOADING

Comenceu per aquí: amb l'estructura i l'autoloading funcionant des del primer dia, la resta de classes es carreguen soles.

📁 Estructura de fitxers obligatòria

```
biblioteca/
├── composer.json
├── .gitignore
├── README.md
├── index.php                    ← únic punt d'entrada (router)
├── views/                       ← una vista per pàgina (no són classes)
│   ├── inici.php
│   ├── materials.php
│   └── ...
└── src/
    ├── Models/
    │   ├── Material.php
    │   ├── Llibre.php
    │   ├── Revista.php
    │   ├── DVD.php
    │   ├── Usuari.php
    │   ├── Prestec.php
    │   └── Biblioteca.php
    ├── Interfaces/
    │   └── Reservable.php
    ├── Traits/
    │   └── Auditoria.php
    ├── Exceptions/
    │   ├── MaterialNoTrobatException.php
    │   ├── MaterialNoDisponibleException.php
    │   ├── MaterialNoPrestatException.php
    │   ├── UsuariNoTrobatException.php
    │   └── EmailInvalidException.php
    └── Data/
        └── DataGenerator.php
```

📦 composer.json

```
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    }
}
```

Després de crear-lo, executeu `composer dump-autoload`. Només cal tornar-ho a fer si modifiqueu el `composer.json`.

🗂️ Namespaces

Cada classe declara el namespace que correspon a la seva carpeta:

Carpeta	Namespace
`src/Models/`	`App\Models`
`src/Interfaces/`	`App\Interfaces`
`src/Traits/`	`App\Traits`
`src/Exceptions/`	`App\Exceptions`
`src/Data/`	`App\Data`

> ⚠️ **Atenció amb les classes natives de PHP.** Dins d'un namespace, `Exception` o `DateTime` es busquen dins el namespace actual. Cal importar-les amb `use` o escriure-les amb barra inicial (`\Exception`, `\DateTime`). És l'error més habitual amb PSR-4.

```php
<?php
namespace App\Exceptions;

use Exception;   // ✅ Sense aquest use, PHP buscaria App\Exceptions\Exception

class EmailInvalidException extends Exception {}
```

🔒 .gitignore

```
/vendor/
```

1️⃣ ESTRUCTURA BASE: ELS MATERIALS

🔹 Classe abstracta Material

Namespace `App\Models`. Usa el trait `Auditoria` (així totes les classes filles l'hereten).

Constructor amb promoted properties: les propietats es declaren directament al constructor. `$disponible` es declara a part perquè no es rep com a paràmetre.

```
<?php
namespace App\Models;

use App\Traits\Auditoria;

abstract class Material
{
    use Auditoria;

    protected bool $disponible = true;

    public function __construct(
        protected int $id,
        protected string $titol,
        protected string $autor,
        protected int $anyPublicacio
    ) {}

    // ... mètodes
}
```

Mètodes abstractes (els han d'implementar les classes filles):

- `calcularMulta(int $diesRetard): float` - Calcula la multa segons els dies de retard
- `getTipus(): string` - Retorna el tipus de material: "Llibre", "Revista" o "DVD"

Mètodes concrets:

- `prestar(): bool` - Si està disponible, el marca com a no disponible, registra l'acció "prestat" i retorna `true`. Si no, retorna `false`
- `retornar(): void` - Marca el material com a disponible i registra l'acció "retornat"
- `__toString(): string` - Retorna la informació en aquest format:

```
[Llibre] #1 - El nom de la rosa (Umberto Eco, 1980) - Disponible
```

Getters obligatoris: `getId(): int`, `getTitol(): string`, `getAutor(): string`, `getAnyPublicacio(): int`, `isDisponible(): bool`

📖 Classe Llibre extends Material implements Reservable

- Propietat privada addicional: `private int $numeroPagines;`
- Constructor: rep els paràmetres de Material + `$numeroPagines` i crida el constructor pare:

```
public function __construct(
    int $id, string $titol, string $autor, int $anyPublicacio,
    int $numeroPagines
) {
    parent::__construct($id, $titol, $autor, $anyPublicacio);
    $this->setNumeroPagines($numeroPagines);   // El setter també valida
```

```
}
```

- `calcularMulta(int $diesRetard): float` - 0.50 € per dia de retard
- `getTipus(): string` - Retorna exactament "Llibre"
- `getNumeroPagines(): int`
- `setNumeroPagines(int $pagines): void` - Si el valor no és > 0, llança `\InvalidArgumentException`

📰 Classe Revista extends Material

- Propietat privada addicional: `private int $numeroEdicio;`
- Constructor: paràmetres de Material + `$numeroEdicio`
- `calcularMulta(int $diesRetard): float` - 0.25 € per dia de retard
- `getTipus(): string` - Retorna exactament "Revista"
- `getNumeroEdicio(): int` i `setNumeroEdicio(int $numero): void` (validació > 0)

> 📝 Les revistes **no** implementen `Reservable`: no es poden reservar.

💿 Classe DVD extends Material implements Reservable

- Propietat privada addicional: `private int $duracio;` (en minuts)
- Constructor: paràmetres de Material + `$duracio`
- `calcularMulta(int $diesRetard): float` - 1.00 € per dia de retard
- `getTipus(): string` - Retorna exactament "DVD"
- `getDuracio(): int` i `setDuracio(int $minuts): void` (validació > 0)
- `getDuracioFormatada(): string` - 150 minuts → "2h 30min"; 45 minuts → "0h 45min"

2️⃣ INTERFÍCIES I TRAITS

🔹 Interfície Reservable

Namespace `App\Interfaces`. Mètodes que han d'implementar les classes:

- `reservar(string $nomUsuari): bool` - Reserva el material per a un usuari. Retorna `false` si ja està reservat
- `cancelarReserva(): bool` - Cancel·la la reserva actual. Retorna `false` si no n'hi ha
- `estaReservat(): bool` - Comprova si el material està reservat
- `getUsuariReserva(): ?string` - Retorna el nom de l'usuari que l'ha reservat, o `null`

Les classes que la implementin necessitaran la propietat `private ?string $usuariReserva = null;` i han de registrar les accions "reservat" i "reserva_cancel·lada".

> 📝 Només **Llibre** i **DVD** implementen aquesta interfície.
>
> 🤔 **Per pensar (opcional, puntua com a millora):** el codi de reserves de Llibre i DVD serà idèntic. Com ho faríeu per no duplicar-lo?

🔹 Trait Auditoria

Namespace `App\Traits`. Propietat privada: `private array $historial = [];`

- `registrarAccio(string $accio, string $detalls = ''): void` - Afegeix una entrada a l'historial amb el format `['data' => 'Y-m-d H:i:s', 'accio' => ..., 'detalls' => ...]`

- `obtenirHistorial(): array` - Retorna tot l'historial
- `obtenirUltimaAccio(): ?array` - Retorna l'última acció, o `null` si no n'hi ha cap
- `netejarHistorial(): void` - Buida l'historial

Accions a registrar: "prestat", "retornat", "reservat", "reserva_cancel·lada".

3️⃣ CLASSE DE GESTIÓ: BIBLIOTECA

🔹 Classe Biblioteca

Propietats privades:

- `private string $nom` (promoted al constructor: `__construct(private string $nom)`)
- `private array $materials = [];` - indexat per l'ID del material
- `private array $usuaris = [];` - indexat per l'ID de l'usuari
- `private array $prestecs = [];` - tots els objectes `Prestec` (actius i retornats)

Gestió de materials:

- `afegirMaterial(Material $material): void`
- `eliminarMaterial(int $id): bool` - Retorna `false` si no existeix
- `cercarPerId(int $id): ?Material`
- `cercarPerTitol(string $titol): array` - Materials que contenen el text (sense distingir majúscules)
- `cercarPerAutor(string $autor): array` - Igual, per autor

Llistats:

- `llistarDisponibles(): array`
- `llistarPrestats(): array`
- `llistarPerTipus(string $tipus): array` - "Llibre", "Revista" o "DVD"
- `llistarPrestecsActius(): array` - Préstecs sense data de retorn
- `llistarReserves(): array` - Materials reservats (cal usar `instanceof Reservable`)
- `llistarPrestecsUsuari(int $usuariId): array` - Tots els préstecs d'un usuari, actius i retornats

Gestió d'usuaris:

- `afegirUsuari(Usuari $usuari): void`
- `cercarUsuari(int|string $identificador): ?Usuari` - **Union type:** si rep un `int`, cerca per ID; si rep un `string`, cerca per nom

Gestió de préstecs i reserves:

- `prestarMaterial(int $materialId, int $usuariId, ?DateTime $dataPrestec = null): Prestec` - Crea el préstec, el guarda a `$prestecs` i l'afegeix a l'usuari. El paràmetre `$dataPrestec` és opcional: si és `null`, s'agafa la data actual. Serveix perquè el `DataGenerator` pugui crear préstecs antics. Llança:
  - `MaterialNoTrobatException` si el material no existeix
  - `UsuariNoTrobatException` si l'usuari no existeix
  - `MaterialNoDisponibleException` si ja està prestat o està reservat per **un altre** usuari. Si el reserva el mateix usuari, es presta i la reserva es cancel·la
- `retornarMaterial(int $materialId): float` - Tanca el préstec i retorna l'import de la multa (0 si no n'hi ha). Llança `MaterialNoPrestatException` si el material no estava prestat



- `reservarMaterial(int $materialId, int $usuariId): bool` - Si el material no és `Reservable`, llança `MaterialNoDisponibleException` amb el missatge "Les revistes no es poden reservar"

Mètode màgic `__call`. Ha de permetre aquestes crides sense definir els mètodes:

- `$biblioteca->getLlibres()`, `getRevistes()`, `getDVDs()` - Retornen només aquell tipus
- `$biblioteca->getMaterials()`, `getUsuaris()`, `getNom()`
- Qualsevol altre mètode inexistent ha de llançar `\BadMethodCallException`

Estadístiques: `obtenirEstadistiques(): array` ha de retornar un array amb aquesta estructura:

```
[
    'total'          => 10,
    'disponibles'    => 7,
    'prestats'       => 3,
    'reservats'      => 1,
    'perTipus'       => ['Llibre' => 4, 'Revista' => 3, 'DVD' => 3],
    'multesPendents' => 3.50
]
```

4️⃣ USUARIS I PRÉSTECS

🔹 Classe Usuari

Propietats privades:

- `private int $id;`
- `private string $nom;`
- `private string $email;`
- `private array $materialsPrestats = [];` - objectes `Material`
- `private \DateTime $dataRegistre;`
- `private array $notificacions = [];` - missatges temporals per mostrar a la web (p. ex. "Préstec realitzat")

Constructor: `__construct(int $id, string $nom, string $email)`. Ha de validar l'email (si no és vàlid, llança `EmailInvalidException`) i inicialitzar `$dataRegistre` amb la data actual.

Mètodes màgics obligatoris:

- `__get(string $propietat): mixed` - Permet llegir `id`, `nom`, `email` i `dataRegistre`. Per a qualsevol altra, retorna `null`
- `__set(string $propietat, mixed $valor): void` - Només permet modificar `email`, amb validació. Per a qualsevol altra propietat, llança `\LogicException`
- `__sleep(): array` - Serialitza totes les propietats **excepte** `$notificacions`
- `__wakeup(): void` - Reinicialitza `$notificacions` com a array buit
- `__toString(): string` - Format: `Anna Puig <anna@mail.com> - 2 préstecs`

Gestió de préstecs i notificacions:

- `afegirPrestec(Material $material): void`
- `eliminarPrestec(int $materialId): void`
- `getMaterialsPrestats(): array`
- `getNumeroMaterialsPrestats(): int`

- `afegirNotificacio(string $missatge): void`
- `obtenirNotificacions(): array` - Les retorna i després les buida

Validació: `private function validarEmail(string $email): bool`

🔹 Classe Prestec

Constructor amb promoted properties i un paràmetre nullable:

```
public function __construct(
    private Material $material,
    private Usuari $usuari,
    private int $diesLimitPrestec = 14,
    ?\DateTime $dataPrestec = null        // Si és null, la data actual
) {
    $this->dataPrestec = $dataPrestec ?? new \DateTime();
}
```

El paràmetre `$dataPrestec` permet crear préstecs antics al DataGenerator, per poder provar retards i multes.

Altres propietats privades: `private \DateTime $dataPrestec;` i `private ?\DateTime $dataRetorn = null;`

Mètodes de gestió:

- `retornar(): void` - Estableix `$dataRetorn` a la data actual
- `estaActiu(): bool` - `true` si encara no s'ha retornat
- `calcularDiesRetard(): int` - Dies que han passat del límit (fins al retorn, o fins avui si encara està actiu). 0 si no n'hi ha
- `calcularMulta(): float` - Usa `calcularMulta()` del material amb els dies de retard
- `estaVencut(): bool` - Comprova si s'ha passat el límit
- `getDiesPendents(): int` - Dies que queden (negatiu si està vençut)

Getters: `getMaterial(): Material`, `getUsuari(): Usuari`, `getDataPrestec(): \DateTime`, `getDataRetorn(): ?\DateTime`, `getDiesLimitPrestec(): int`

5️⃣ EXCEPCIONS PERSONALITZADES

Namespace `App\Exceptions`. Totes estenen `Exception` (recordeu el `use Exception;`).

Excepció	Quan es llança
`MaterialNoTrobatException`	L'ID del material no existeix
`MaterialNoDisponibleException`	El material està prestat, reservat per un altre usuari, o no es pot reservar
`MaterialNoPrestatException`	Es vol retornar un material que no estava prestat
`UsuariNoTrobatException`	L'usuari no existeix
`EmailInvalidException`	Email amb format incorrecte

`MaterialNoDisponibleException` és la més completa:

- Propietat `private int $materialId;`
- Constructor que rep `$materialId` i un missatge opcional

- `getMaterialId(): int`
- `__toString(): string` amb missatge personalitzat

Teniu la teoria i aquest exemple resolt a l'**annex** del final.

6️⃣ DADES I PERSISTÈNCIA

🔹 DataGenerator

Classe `App\Data\DataGenerator` amb un mètode estàtic `crear(): Biblioteca` que retorni una biblioteca amb, com a mínim:

- 4 llibres, 3 revistes i 3 DVDs amb dades reals
- 4 usuaris
- 2 préstecs actius, un dels quals **vençut** (creat amb una data antiga)
- 1 reserva

🔹 Persistència amb la sessió

Sense base de dades, els objectes es perden a cada petició. Per mantenir els canvis (préstecs, retorns, reserves) entre pàgines, guardarem la biblioteca a la sessió. PHP la **serialitza automàticament**: aquí és on entren en joc `__sleep()` i `__wakeup()`.

```
<?php
// index.php
require_once __DIR__ . '/vendor/autoload.php';   // ⚠️ Sempre ABANS de session_start()

use App\Data\DataGenerator;

session_start();

if (!isset($_SESSION['biblioteca'])) {
    $_SESSION['biblioteca'] = DataGenerator::crear();
}

$biblioteca = $_SESSION['biblioteca'];
```

> ⚠️ L'autoloader s'ha de carregar abans de `session_start()`. Si no, PHP no sap quines classes té els objectes guardats i no els pot reconstruir.

Afegiu una pàgina `index.php?page=reset` que faci `session_destroy()` i torni a l'inici, per poder regenerar les dades.

7️⃣ INTERFÍCIE WEB

🔹 Router amb match

`index.php` és l'únic punt d'entrada. Decideix quina vista carregar amb una expressió `match` i captura les excepcions perquè l'usuari mai vegi un *Fatal error*:

```
$page = $_GET['page'] ?? 'inici';

try {
    $vista = match ($page) {
        'inici'     => 'views/inici.php',
        'materials' => 'views/materials.php',
```

```
        'material'   => 'views/material.php',
        // ... la resta de pàgines
        default      => 'views/404.php',
    };
    require $vista;
} catch (MaterialNoDisponibleException $e) {
    $error = (string) $e;           // Usa el __toString() personalitzat
    require 'views/error.php';
} catch (Exception $e) {
    $error = $e->getMessage();
    require 'views/error.php';
}
```

🌐 URLs operatives

🏠 Pàgina principal

- `index.php` - Dashboard amb resum del sistema

📚 Materials

- `index.php?page=materials` - Llistat de tots els materials en una taula
- `index.php?page=materials&type=llibre` - Només llibres (també `type=revista` i `type=dvd`)
- `index.php?page=material&id=X` - Detall d'un material, amb el seu historial d'auditoria

👥 Usuaris

- `index.php?page=usuaris` - Llistat d'usuaris en una taula
- `index.php?page=usuari&id=X` - Perfil de l'usuari amb els préstecs actius
- `index.php?page=usuari&id=X&action=prestecs` - Historial de préstecs de l'usuari

🔄 Préstecs

- `index.php?page=prestec&material_id=X&usuari_id=Y` - Realitzar un préstec
- `index.php?page=retorn&material_id=X` - Retornar un material (mostrant la multa, si n'hi ha)
- `index.php?page=prestecs` - Llistat de préstecs actius, destacant els vençuts

📝 Reserves

- `index.php?page=reserva&material_id=X&usuari_id=Y` - Crear una reserva
- `index.php?page=reserves` - Llistat de reserves actives

📊 Estadístiques i auditoria

- `index.php?page=estadistiques` - Dashboard amb les estadístiques
- `index.php?page=auditoria` - Historial d'accions de tots els materials

🔧 Demostracions tècniques

- `index.php?page=demo_magic` - `__toString`, `__call`, `__get` i `__set` (incloent-hi un `__set` no permès)
- `index.php?page=demo_serialization` - Mostra el `serialize()` d'un usuari i comprova que `$notificacions` no s'hi guarda
- `index.php?page=demo_exceptions` - Provoca cadascuna de les 5 excepcions i les captura
- `index.php?page=demo_autoload` - Mostra les classes carregades (`get_declared_classes()` filtrant per `App\`)

- `index.php?page=reset` - Reinicia les dades de la sessió

🎨 Format de visualització

```
<!-- Taules per a llistats -->
<table class="table">
    <thead>
        <tr><th>ID</th><th>Nom</th><th>Estat</th><th>Accions</th></tr>
    </thead>
    <tbody>
        <!-- Dades dinàmiques -->
    </tbody>
</table>

<!-- Llistes per a detalls -->
<ul class="list-group">
    <li class="list-item">Propietat: Valor</li>
</ul>

<!-- Targetes per a estadístiques -->
<div class="stats-grid">
    <div class="stat-card">
        <h3>Total materials</h3>
        <span class="number">10</span>
    </div>
</div>
```

🎯 Navegació principal

```
<nav class="navbar">
    <a href="index.php">🏠 Inici</a>
    <a href="index.php?page=materials">📚 Materials</a>
    <a href="index.php?page=usuaris">👥 Usuaris</a>
    <a href="index.php?page=prestecs">🔄 Préstecs</a>
    <a href="index.php?page=reserves">📝 Reserves</a>
    <a href="index.php?page=estadistiques">📊 Estadístiques</a>
    <a href="index.php?page=auditoria">🗂️ Auditoria</a>
</nav>
```

8️⃣ CARACTERÍSTIQUES DE PHP 8

Característica	On s'ha d'utilitzar (com a mínim)
Typed properties	Totes les propietats han de tenir tipus
Promoted properties	Constructors de `Material`, `Biblioteca` i `Prestec`
Nullable types	`getUsuariReserva(): ?string`, `cercarPerId(): ?Material`, `$dataRetorn`
Union types	`cercarUsuari(int|string $identificador)`
`mixed`	`__get()` i `__set()`
`match`	Router de l'`index.php` i filtre per `type` dels materials

✅ CRITERIS D'AVALUACIÓ

Concepte	Pes	Descripció
Classes i herència	20%	Material abstracta, mètodes abstractes, herència correcta, promoted properties i crida a `parent::__construct()`
Visibilitat i encapsulació	10%	Ús correcte de public/protected/private, getters i setters amb validació, typed properties
Interfícies i traits	15%	Reservable només a Llibre i DVD, ús d'`instanceof`, trait Auditoria funcional a tots els materials
Mètodes màgics	15%	`__toString`, `__call`, `__get`, `__set`, `__sleep` i `__wakeup` amb lògica adequada
Gestió d'errors	10%	Les 5 excepcions, try/catch a la interfície, cap error fatal visible
Autoloading i estructura	10%	Composer amb PSR-4, namespaces coherents amb les carpetes, estructura de fitxers correcta
Interfície web i persistència	15%	Totes les URLs operatives, router amb `match`, dades persistents a la sessió
Codi i README	5%	Codi net i comentat, README complet

📝 ENTREGA I DEMOSTRACIÓ

Es lliura **un sol link al repositori Git** a la tasca del Moodle. El link del vídeo ha d'estar dins del README.

🗃️ Repositori Git

- **Accés:** repositori públic, o privat amb invitació a la professora
- **Estructura:** tots els fitxers segons l'estructura obligatòria
- **Sense `vendor/`:** ha d'estar al `.gitignore` (es regenera amb `composer install`)
- **Historial:** mínim 5 commits fets en dies diferents, amb missatges que expliquin el canvi (p. ex. "Afegeix trait Auditoria"). No s'accepta un únic commit amb tot el projecte
- **Data límit:** es corregeix l'últim commit anterior a la data d'entrega
- **Codi comentat:** cada classe amb un comentari que expliqui què fa, i els mètodes no evidents amb un comentari breu
- **`README.md`** amb:
  - Instruccions d'instal·lació: `composer install`
  - Instruccions d'execució: `php -S localhost:8000`
  - Llista de les URLs i què fa cadascuna
  - Decisions de disseny o millores que hàgiu fet
  - **Link al vídeo** de demostració

🎬 Vídeo de demostració

Durada: 2 minuts aproximadament. **Format:** gravació de pantalla **amb veu**. **On penjar-lo:** YouTube (no llistat) o Google Drive amb accés per al compte de l'institut. No el pugeu al repositori.

El vídeo ha de seguir **aquest guió, en aquest ordre**. Si falta algun pas, aquell apartat no es pot avaluar.

#	Què heu de mostrar	Temps
1	Al terminal: `composer install` i `php -S localhost:8000`. Obriu `index.php` al navegador	10"
2	**Materials:** llistat complet i filtre per `type=dvd`. Obriu el detall d'un material	15"
3	**Préstec:** presteu un llibre a un usuari i mostreu el perfil de l'usuari amb el préstec actiu	15"
4	**Excepció:** intenteu prestar el mateix llibre a un altre usuari i mostreu el missatge d'error	10"
5	**Retorn amb multa:** retorneu el préstec vençut del DataGenerator i mostreu la multa	10"
6	**Reserves:** reserveu un DVD i intenteu reservar una revista (excepció)	15"
7	**Estadístiques i auditoria:** mostreu les dues pàgines	10"
8	**Demos:** `demo_magic` i `demo_serialization`	15"
9	**Codi:** obriu el `__call` de `Biblioteca` i expliqueu amb les vostres paraules com funciona	20"

> 📝 No cal fer cap introducció ni explicar què és el projecte: comenceu directament pel pas 1. Abans de gravar, feu `page=reset` perquè les dades siguin les inicials.

🚨 ANNEX: EXCEPCIONS PERSONALITZADES EN PHP

📚 Què són les excepcions?

Les excepcions són un mecanisme per gestionar errors de manera controlada, sense que el programa s'aturi de cop. A més, ens diuen **per què** ha fallat una operació.

Sense excepcions (problemàtic)

```
public function prestarMaterial(int $id): bool
{
    $material = $this->cercarPerId($id);
    if ($material === null || !$material->isDisponible()) {
        return false;   // ❌ No sabem per què ha fallat
    }
    // ...
}
```

Amb excepcions (millor)

```
public function prestarMaterial(int $id, int $usuariId): Prestec
{
    $material = $this->cercarPerId($id);
    if ($material === null) {
        throw new MaterialNoTrobatException("No existeix cap material amb ID $id");
    }
    if (!$material->isDisponible()) {
        throw new MaterialNoDisponibleException($id, "El material ja està prestat");
    }
    // ...
}
```

🏗️ Estructura de la classe Exception nativa

Versió simplificada del que hereten les nostres excepcions:

```
class Exception
{
    protected string $message;   // Missatge d'error
    protected int $code;         // Codi d'error
    protected string $file;      // Fitxer on s'ha produït
    protected int $line;         // Línia on s'ha produït

       public function __construct(string $message = "", int $code = 0, ?Throwable
$previous = null)
    public function getMessage(): string
    public function getCode(): int
    public function getFile(): string
    public function getLine(): int
    public function __toString(): string
}
```

> 📝 Els mètodes `getMessage()`, `getCode()`, etc. són `final`: no es poden sobreescriure. El que sí podem sobreescriure és el constructor i `__toString()`.

🎯 Exemple resolt: MaterialNoDisponibleException

```
<?php
namespace App\Exceptions;

use Exception;

class MaterialNoDisponibleException extends Exception
{
    private int $materialId;

       public function __construct(int $materialId, string $message = "Material no
disponible")
    {
        $this->materialId = $materialId;
        parent::__construct($message);
    }

    public function getMaterialId(): int
    {
        return $this->materialId;
    }

    public function __toString(): string
    {
            return "Error: el material amb ID {$this->materialId} no està disponible.
{$this->getMessage()}";
    }
}
```

🧲 Capturar-les

```
try {
    $biblioteca->prestarMaterial(5, 2);
    echo "Préstec realitzat";
} catch (MaterialNoTrobatException $e) {
    echo "No existeix: " . $e->getMessage();
} catch (MaterialNoDisponibleException $e) {
    echo $e;                                    // Crida __toString()
    echo "ID afectat: " . $e->getMaterialId();
} catch (Exception $e) {
    echo "Error inesperat: " . $e->getMessage();
}
```

⚠️ L'ordre dels `catch` importa: sempre de la més **específica** a la més **general**. Si `catch (Exception $e)` anés primer, capturaria totes les altres.